

Clase 230 — Seguridad de Infrastructure as Code (Terraform)

Parte: **10 — Seguridad en la nube y contenedores** · Fuente: HashiCorp Terraform docs y OWASP Infrastructure as Code Security 🕒 Duración estimada: **120 min** · Nivel: **Intermedio**

🎯 Objetivo

Aplicar seguridad al código que define la infraestructura. El alumno aprenderá a escanear configuraciones Terraform en busca de misconfiguraciones (tfsec/Checkov), a proteger el estado (`state`) que contiene datos sensibles, a evitar secretos en el código y a integrar estos controles en el pipeline para detectar problemas antes de desplegar.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Escanear** código Terraform con tfsec y Checkov e interpretar hallazgos.
- 2. **Proteger** el `state` remoto (cifrado, bloqueo, acceso restringido).
- 3. **Evitar** secretos en el código y en el state mediante gestores de secretos.
- 4. **Integrar** validación de seguridad IaC en un pipeline CI (`plan` + escaneo + policy).
- 5. **Aplicar** policy-as-code para bloquear despliegues inseguros.

📖 Temas

#	Tema	Por qué importa
1	IaC y drift	Reproducibilidad y coherencia con lo desplegado
2	EscanEO estático (tfsec/Checkov)	Detectar misconfig antes de aplicar
3	Gestión del state	Contiene secretos y estado sensible
4	Secretos en Terraform	Nunca en texto plano ni en el state
5	Módulos y proveedores confiables	Cadena de suministro de IaC
6	Policy-as-code (OPA/Sentinel)	Guardarraíles automáticos
7	IaC en el pipeline CI/CD	Shift-left de la seguridad de infra

📖 Definiciones y características

- **Infrastructure as Code (IaC):** definir infraestructura en archivos versionados. *Clave:* auditable y reproducible, pero un error se replica a escala.
- **Terraform state:** archivo con el mapeo entre código y recursos reales. *Clave:* puede contener secretos en texto plano; protéjelo.

- **Drift:** divergencia entre el código y la infraestructura real. *Clave:* `terraform plan` lo detecta.
- **tfsec / Checkov:** escáneres estáticos de IaC. *Clave:* encuentran buckets públicos, SG abiertos, cifrado ausente.
- **Policy-as-code:** reglas (OPA/Rego, Sentinel) que aprueban o bloquean planes. *Clave:* impide desplegar lo no conforme.
- **Backend remoto:** almacenamiento del state (S3+DynamoDB, GCS, Terraform Cloud). *Clave:* habilita cifrado y bloqueo concurrente.
- **Módulo:** paquete reutilizable de Terraform. *Clave:* verifica origen y versión para la cadena de suministro.

Herramientas y preparación

- **Terraform** instalado y un proveedor de laboratorio configurado.
- **tfsec**, **Checkov** y **terrascan** para escaneo estático.
- **OPA/Conftest** para policy-as-code.

```
# Escaneo estático de un directorio Terraform
tfsec .
checkov -d .
# Validar un plan contra políticas OPA
terraform plan -out plan.tfplan && terraform show -json plan.tfplan > plan.json
conftest test plan.json
```

Laboratorio guiado

1. Escribe un módulo Terraform inseguro: un bucket con `acl = "public-read"`, un security group con `0.0.0.0/0` en el puerto 22 y un recurso sin cifrado.
2. Ejecuta `tfsec .` y `checkov -d .`; anota los identificadores de cada hallazgo y su severidad.
3. Corrige el código: bloquea acceso público, restringe el SG a un rango, activa cifrado en reposo. Reejecuta los escáneres hasta 0 hallazgos altos.
4. Configura un **backend remoto** cifrado con bloqueo (por ejemplo S3 + DynamoDB) y verifica que el state ya no queda en local sin protección.
5. Elimina cualquier secreto del código; inyéctalo desde variables de entorno o un gestor de secretos y comprueba que no aparece en el `.tf`.
6. Escribe una política **OPA/Conftest** que rechace planes con recursos sin cifrado o con acceso público, e intégrala tras `terraform plan`.
7. Simula el pipeline: `fmt` → `validate` → `tfsec / checkov` → `plan` → `conftest` → (aprobación) → `apply`. Falla el pipeline si el escaneo encuentra un hallazgo crítico.

Ejercicios

1. Corrige un hallazgo concreto de tfsec y documenta la regla que lo detectó.
2. Migra un state local a un backend remoto cifrado con bloqueo.
3. Escribe una política Rego que exija etiquetas obligatorias en todos los recursos.
4. Detecta drift introduciendo un cambio manual y ejecutando `terraform plan`.
5. Fija (pin) la versión de un módulo y de un proveedor y explica por qué.
6. Integra el escaneo IaC como paso obligatorio en un workflow de CI.

Reto verificable

Toma un repositorio Terraform con misconfiguraciones y déjalo "verde": sin hallazgos críticos en tfsec/Checkov, con state remoto cifrado y bloqueado, sin secretos en el código, y con una policy OPA que bloquea recursos inseguros en el pipeline.

Criterio de aceptación: tfsec y checkov reportan 0 hallazgos críticos/altos, conftest rechaza un plan que reintroduzca un bucket público, y no hay ningún secreto en texto plano en los .tf ni en el backend.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Secreto visible en terraform.tfstate	El recurso guardó el valor en el state; usa un gestor de secretos y cifra el backend.
tfsec pasa pero el recurso sigue inseguro	Regla suprimida con #tfsec:ignore ; revisa las supresiones injustificadas.
Error acquiring the state lock	Otro apply en curso o lock huérfano; espera o libera con force-unlock con cuidado.
Drift no detectado	Cambios fuera de Terraform; ejecuta plan periódicamente y usa detección de drift.
Módulo malicioso o desactualizado	Origen no verificado; fija versión y usa fuentes confiables.

Preguntas frecuentes

? ¿Por qué el state de Terraform es sensible? Porque puede contener valores en texto plano (contraseñas de bases de datos, claves) y el mapa completo de tu infraestructura. Debe cifrarse en reposo, tener acceso restringido y bloqueo para evitar corrupción.

? ¿tfsec o Checkov? Ambos son buenos; suelen usarse juntos porque sus reglas no coinciden al 100%. Checkov cubre más frameworks (Terraform, CloudFormation, Kubernetes) y tfsec es muy rápido para Terraform. Ejecuta los dos en CI.

? ¿El escaneo estático reemplaza al CSPM? No. El escaneo IaC detecta problemas *antes* de desplegar (shift-left); el CSPM (clase 231) evalúa lo *ya desplegado* en tiempo de ejecución, incluyendo cambios hechos fuera de Terraform. Se complementan.

Referencias

- Terraform docs — Backends y state. <https://developer.hashicorp.com/terraform/language/state>
- tfsec. <https://github.com/aquasecurity/tfsec>
- Checkov. <https://www.checkov.io/>
- Open Policy Agent / Conftest. <https://www.openpolicyagent.org/>
- OWASP Infrastructure as Code Security Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Infrastructure_as_Code_Security_Cheat_Sheet.html

Siguiente clase

Clase 231 - Cloud Security Posture Management (CSPM)